



Verifying Solidity Smart Contracts Via Communication Abstraction in SmartACE

Scott Wesley



Jorge Navas



Valentin Wüstholtz



Maria Christakis



Arie Gurfinkel
Richard Treffer



Motivation and Background:

The Problem of Smart Contract Verification

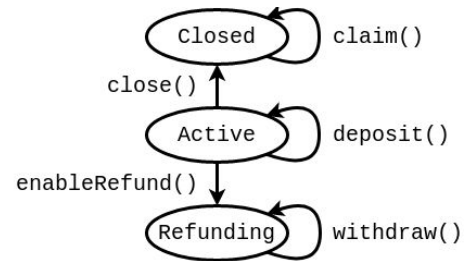


Motivation: What is a smart contract

- **Smart contracts** are finite state machines that manage assets on a blockchain.

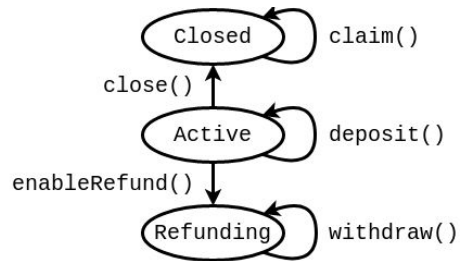
Motivation: What is a smart contract

- **Smart contracts** are finite state machines that manage assets on a blockchain.
- Often smart contracts describe **financial agreements**, such as an escrow service.



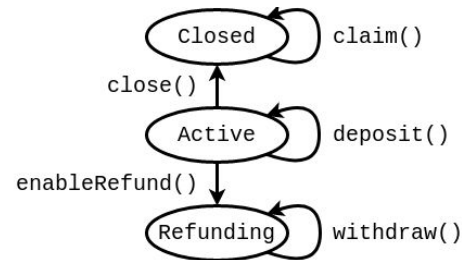
Motivation: What is a smart contract

- **Smart contracts** are finite state machines that manage assets on a blockchain.
- Often smart contracts describe **financial agreements**, such as an escrow service.
- **Solidity** smart contracts facilitate **information flow** (e.g., *making deposits*) between many users with **unique addresses**.



Motivation: What is a smart contract

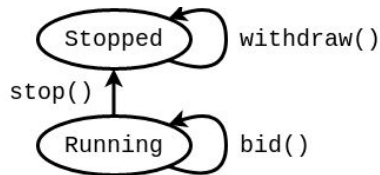
- **Smart contracts** are finite state machines that manage assets on a blockchain.
- Often smart contracts describe **financial agreements**, such as an escrow service.
- **Solidity** smart contracts facilitate **information flow** (e.g., *making deposits*) between many users with **unique addresses**.
- For example, the **program given to the right** is the interface for a Solidity smart contract that implements an **escrow service with a designated owner**.



```
1. contract RefundEscrow {
2.     constructor(address beneficiary) public { ... }
3.     function deposit() public payable { ... }
4.     function withdraw() public { ... }
5.     function close() public { ... }
6.     function enableRefund() public { ... }
7.     function claim() public { ... }
8. }
```

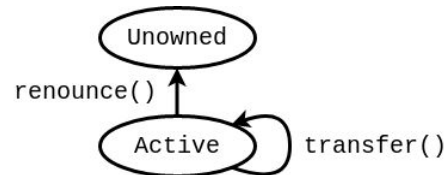
Motivation: Examples of Smart Contracts

Open Bid Auction Contract



```
1. contract Auction {  
2.     constructor(address manager) public { ... }  
3.     function bid() public payable { ... }  
4.     function withdraw() public { /... }  
5.     function stop() public { ... }  
6. }
```

Ownership Interface Contract



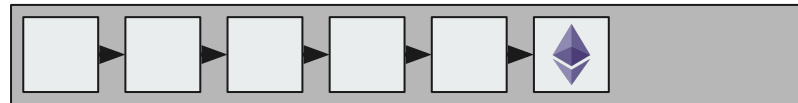
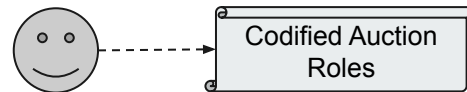
```
1. contract Ownable {  
2.     constructor() public { ... }  
3.     function transfer(address owner) public { ... }  
4.     function renounce() public { ... }  
5. }
```



Motivation: Trust via Source Code Verification

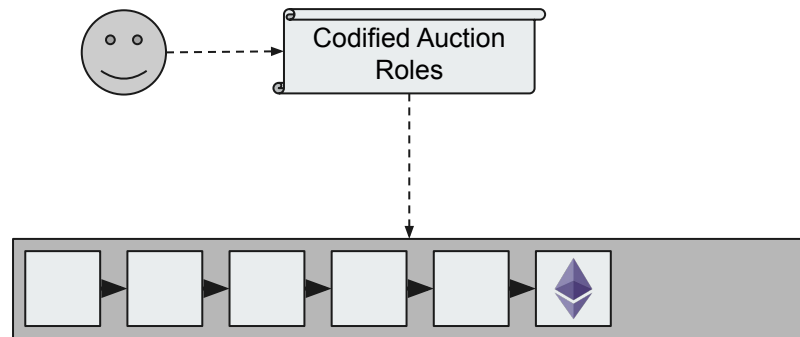
Motivation: Trust via Source Code Verification

1. A developer codifies a financial agreement (e.g., *an auction*) as a Solidity smart contract.



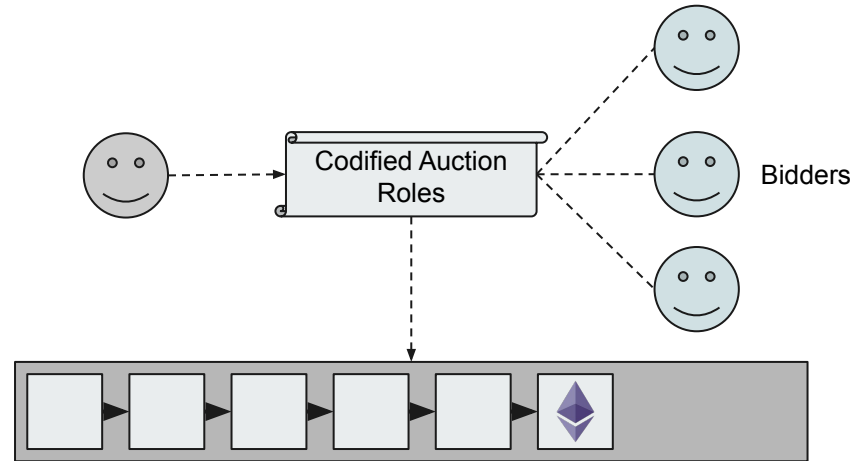
Motivation: Trust via Source Code Verification

1. A developer codifies a financial agreement (e.g., *an auction*) as a Solidity smart contract.
2. The smart contract is deployed to the Ethereum blockchain as immutable code.



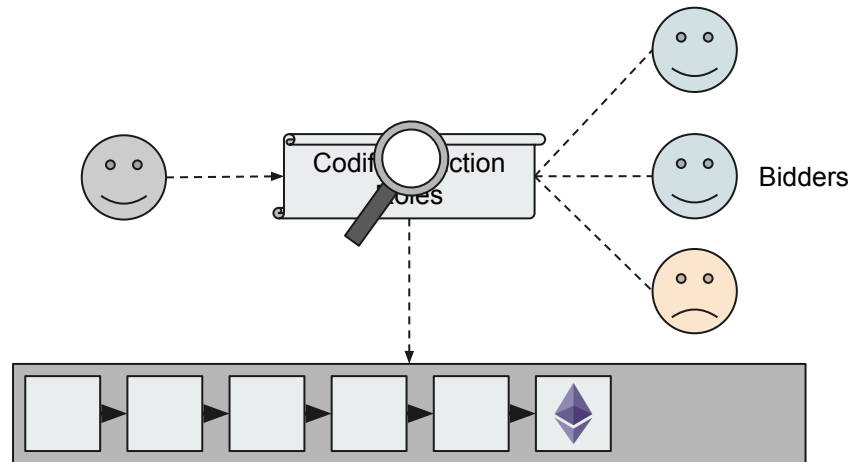
Motivation: Trust via Source Code Verification

1. A developer codifies a financial agreement (e.g., an auction) as a Solidity smart contract.
2. The smart contract is deployed to the Ethereum blockchain as immutable code.
3. Untrusting parties interact via the contract as a mechanism to enforce cooperation.



Motivation: Trust via Source Code Verification

1. A developer codifies a financial agreement (e.g., an auction) as a Solidity smart contract.
2. The smart contract is deployed to the Ethereum blockchain as immutable code.
3. Untrusting parties interact via the contract as a mechanism to enforce cooperation.
4. To ensure that trust can be placed in the contract, each party **must verify it is correct against all possible transaction sequences.**





Notation: States and Traces

```
01. contract Auction {
02.   mapping(address => uint) bids;
03.   address manager; uint leadingBid; bool stopped;
04.
05.   function bid() public payable {
06.     // Checks permission to place bid.
07.     require(!stopped, "Auction must be ongoing.");
08.     require(msg.sender != manager, "Bidder must not be manager.");
09.     // Checks bid is valid.
10.     uint curBid = bids[msg.sender];
11.     require(curBid == 0 || curBid < leadingBid, "Leading bidder.");
12.     require(curBid + msg.value > leadingBid, "Bid too small.");
13.     // Records bid.
14.     bids[msg.sender] = curBid + msg.value;
15.     leadingBid = bids[msg.sender];
16.   }
17.
18.   constructor(address manager) public { /* ... */ }
19.   function withdraw() public { /* ... */ }
20.   function stop() public { /* ... */ }
21. }
```

Notation: States and Traces

```
01. contract Auction {
02.   mapping(address => uint) bids;
03.   address manager; uint leadingBid; bool stopped;
04.
05.   function bid() public payable {
06.     // Checks permission to place bid.
07.     require(!stopped, "Auction must be ongoing.");
08.     require(msg.sender != manager, "Bidder must not be manager.");
09.     // Checks bid is valid.
10.     uint curBid = bids[msg.sender];
11.     require(curBid == 0 || curBid < leadingBid, "Leading bidder.");
12.     require(curBid + msg.value > leadingBid, "Bid too small.");
13.     // Records bid.
14.     bids[msg.sender] = curBid + msg.value;
15.     leadingBid = bids[msg.sender];
16.   }
17.
18.   constructor(address manager) public { /* ... */ }
19.   function withdraw() public { /* ... */ }
20.   function stop() public { /* ... */ }
21. }
```

manager leadingBid stopped

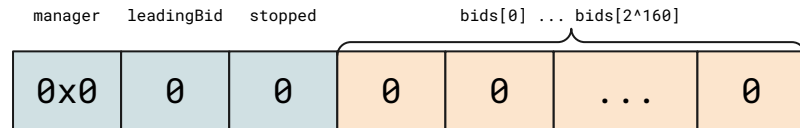
0x0	0	0
-----	---	---

Notation: States and Traces

```

01. contract Auction {
02.     mapping(address => uint) bids;
03.     address manager; uint leadingBid; bool stopped;
04.
05.     function bid() public payable {
06.         // Checks permission to place bid.
07.         require(!stopped, "Auction must be ongoing.");
08.         require(msg.sender != manager, "Bidder must not be manager.");
09.         // Checks bid is valid.
10.         uint curBid = bids[msg.sender];
11.         require(curBid == 0 || curBid < leadingBid, "Leading bidder.");
12.         require(curBid + msg.value > leadingBid, "Bid too small.");
13.         // Records bid.
14.         bids[msg.sender] = curBid + msg.value;
15.         leadingBid = bids[msg.sender];
16.     }
17.
18.     constructor(address manager) public { /* ... */ }
19.     function withdraw() public { /* ... */ }
20.     function stop() public { /* ... */ }
21. }

```

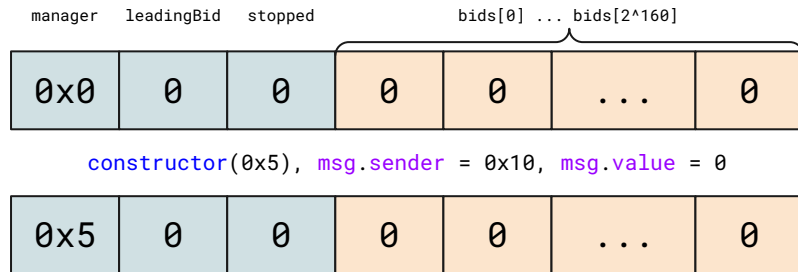


Notation: States and Traces

```

01. contract Auction {
02.   mapping(address => uint) bids;
03.   address manager; uint leadingBid; bool stopped;
04.
05.   function bid() public payable {
06.     // Checks permission to place bid.
07.     require(!stopped, "Auction must be ongoing.");
08.     require(msg.sender != manager, "Bidder must not be manager.");
09.     // Checks bid is valid.
10.     uint curBid = bids[msg.sender];
11.     require(curBid == 0 || curBid < leadingBid, "Leading bidder.");
12.     require(curBid + msg.value > leadingBid, "Bid too small.");
13.     // Records bid.
14.     bids[msg.sender] = curBid + msg.value;
15.     leadingBid = bids[msg.sender];
16.   }
17.
18.   constructor(address manager) public { /* ... */ }
19.   function withdraw() public { /* ... */ }
20.   function stop() public { /* ... */ }
21. }

```

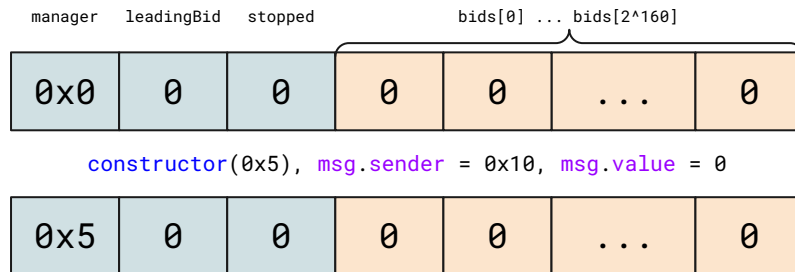


Notation: States and Traces

```

01. contract Auction {
02.   mapping(address => uint) bids;
03.   address manager; uint leadingBid; bool stopped;
04.
05.   function bid() public payable {
06.     // Checks permission to place bid.
07.     require(!stopped, "Auction must be ongoing.");
08.     require(msg.sender != manager, "Bidder must not be manager.");
09.     // Checks bid is valid.
10.     uint curBid = bids[msg.sender];
11.     require(curBid == 0 || curBid < leadingBid, "Leading bidder.");
12.     require(curBid + msg.value > leadingBid, "Bid too small.");
13.     // Records bid.
14.     bids[msg.sender] = curBid + msg.value;
15.     leadingBid = bids[msg.sender];
16.   }
17.
18.   constructor(address manager) public { /* ... */ }
19.   function withdraw() public { /* ... */ }
20.   function stop() public { /* ... */ }
21. }

```

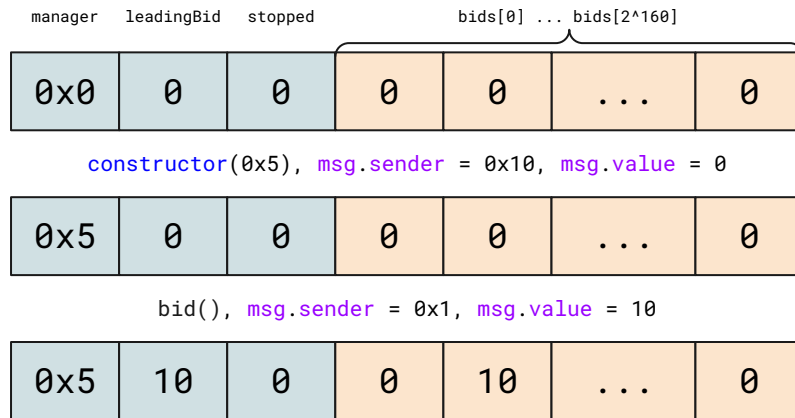


Notation: States and Traces

```

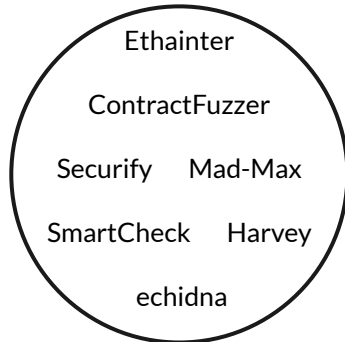
01. contract Auction {
02.     mapping(address => uint) bids;
03.     address manager; uint leadingBid; bool stopped;
04.
05.     function bid() public payable {
06.         // Checks permission to place bid.
07.         require(!stopped, "Auction must be ongoing.");
08.         require(msg.sender != manager, "Bidder must not be manager.");
09.         // Checks bid is valid.
10.         uint curBid = bids[msg.sender];
11.         require(curBid == 0 || curBid < leadingBid, "Leading bidder.");
12.         require(curBid + msg.value > leadingBid, "Bid too small.");
13.         // Records bid.
14.         bids[msg.sender] = curBid + msg.value;
15.         leadingBid = bids[msg.sender];
16.     }
17.
18.     constructor(address manager) public { /* ... */ }
19.     function withdraw() public { /* ... */ }
20.     function stop() public { /* ... */ }
21. }

```



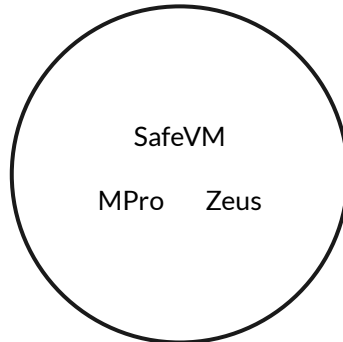
Challenges: Current Solutions

Declarative Static Analysis & Dynamic Analysis



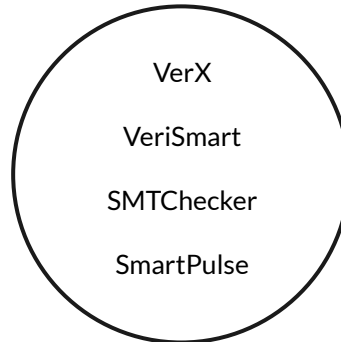
(+) Highly Applicable
(-) Limited to Bug Finding

Model Checking & Symbolic Execution (*Pre/Post*)



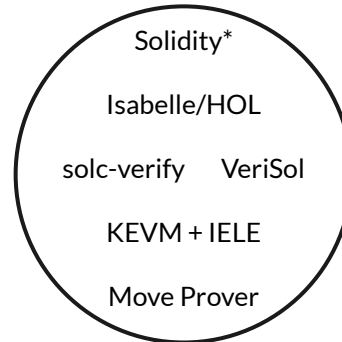
(+) Moderate Applicability
(-) Shallow Properties

Model Checking & Symbolic Execution (*Contract Invariants*)



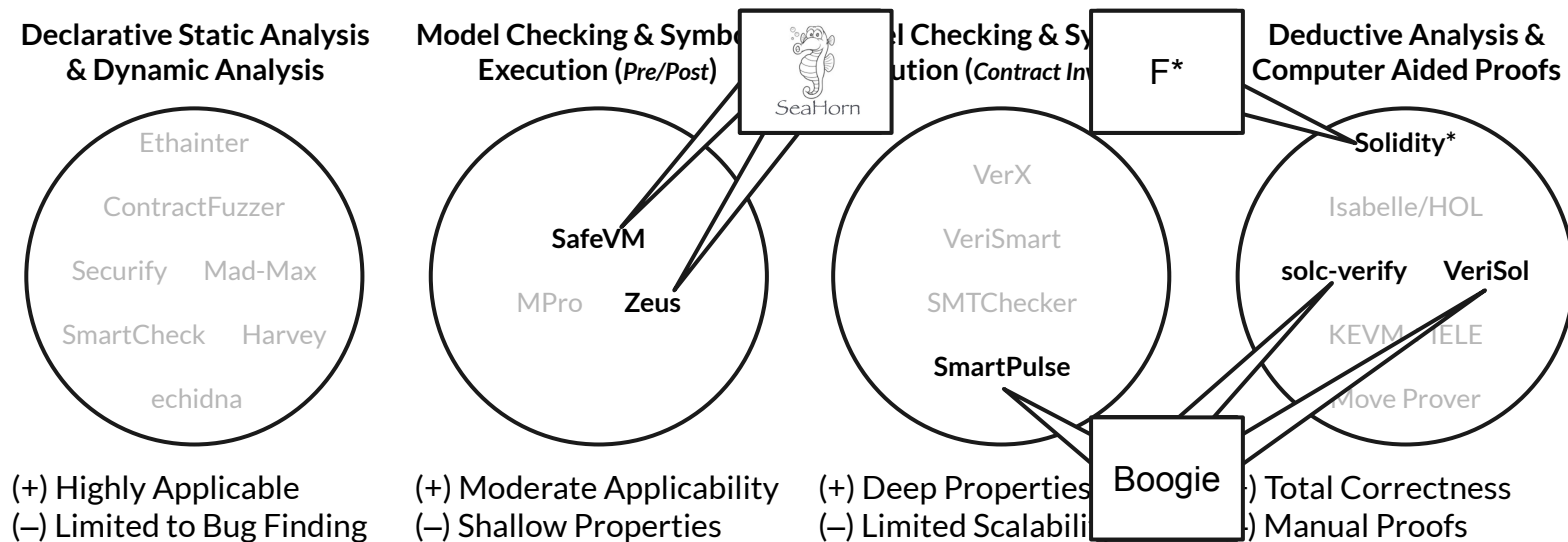
(+) Deep Properties
(-) Limited Scalability

Deductive Analysis & Computer Aided Proofs

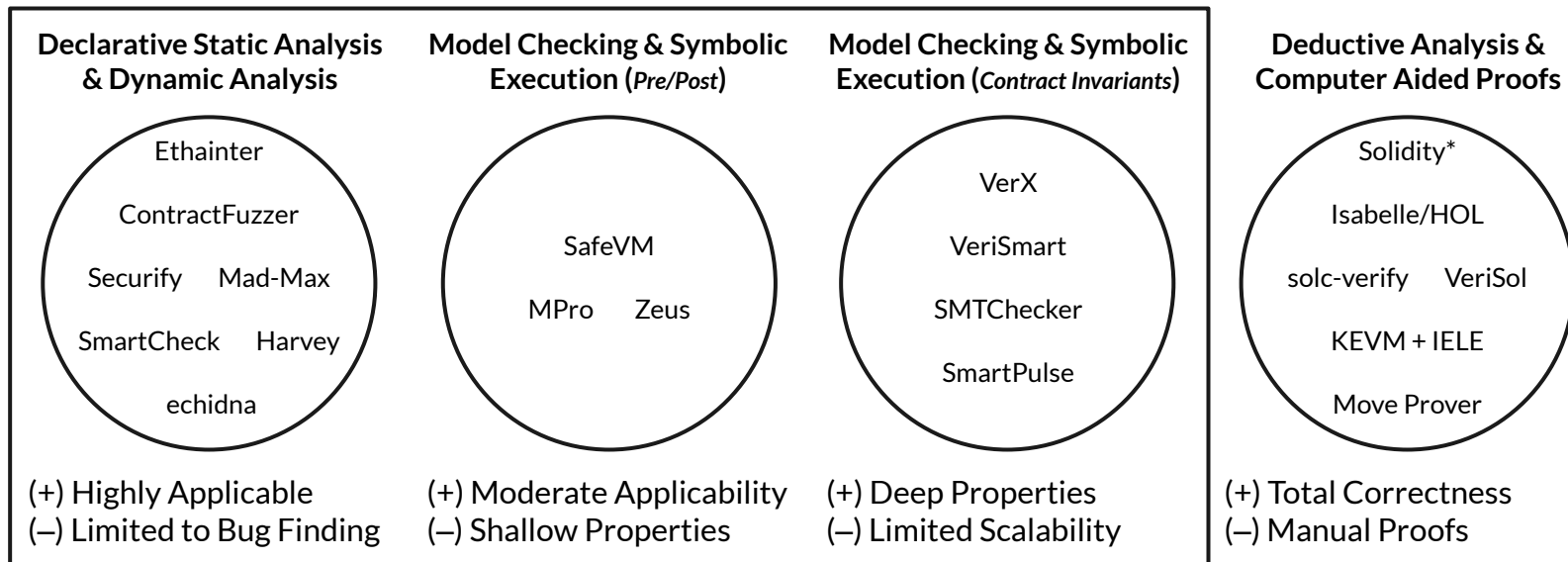


(+) Total Correctness
(-) Manual Proofs

Challenges: Current Solutions



Challenges: Current Solutions

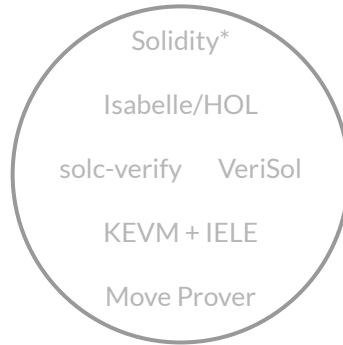


Challenges: Current Solutions

SmartACE is designed with the following goals in mind.

1. **Inter-Transactional Analysis:** SmartACE verifies contract invariants.
2. **Support for Multiple Off-the-Shelf Analyzers:** SmartACE can integrate any LLVM analyzer regardless of its analysis technique.
3. **Agnostic to Verification Decisions:** SmartACE uses a modular design that separates analysis choices (e.g., *handling overflow or non-linear arithmetic*) from the problem encoding.
4. **Support for Local Reasoning:** SmartACE is the first Solidity analyzer to support local reasoning. Our SAS 2021 paper* shows that local reasoning could outperform VerX by up to 10x on the Verx Benchmark set.

Deductive Analysis &
Computer Aided Proofs



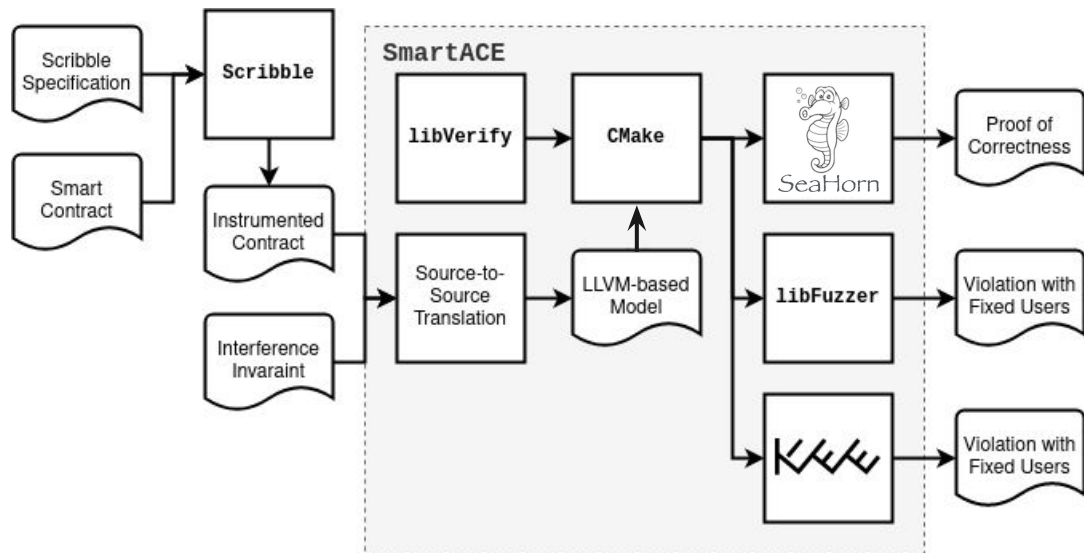
(+) Total Correctness

(-) Manual Proofs

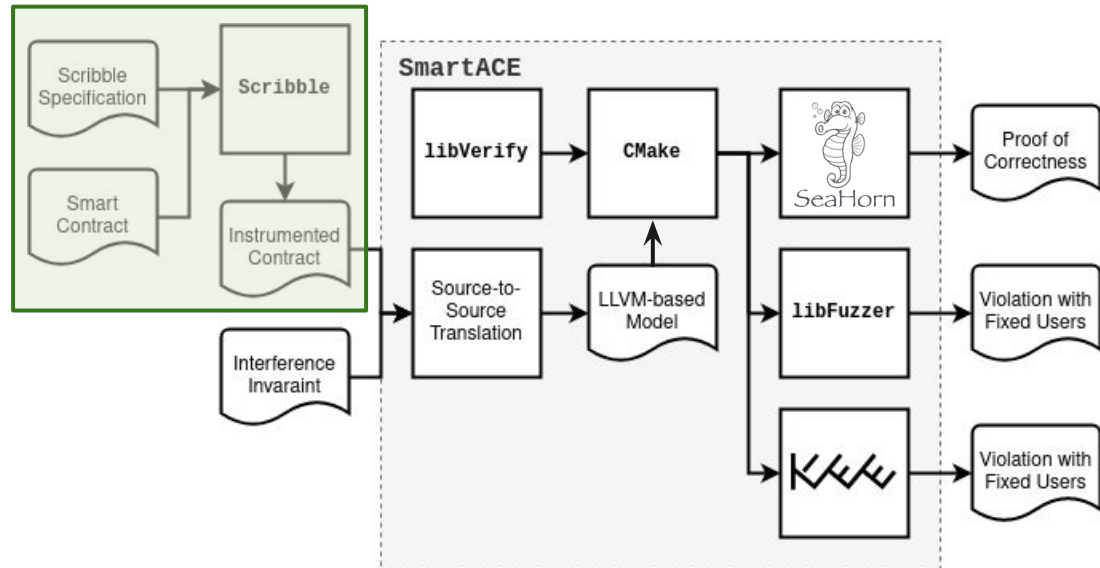
* Wesley et al., “Compositional verification of smart contracts through communication abstraction”, 28th International Symposium in Static Analysis, 2021, pp. 429–452.

Program Analysis in SmartACE: Architecture and Techniques

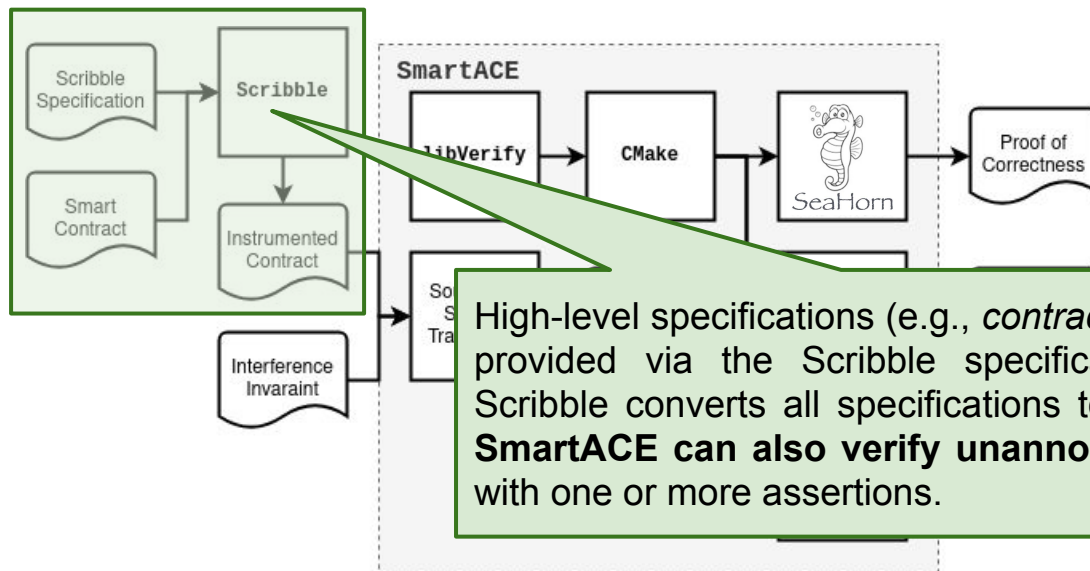
Overview: SmartACE Architecture



Overview: SmartACE Architecture

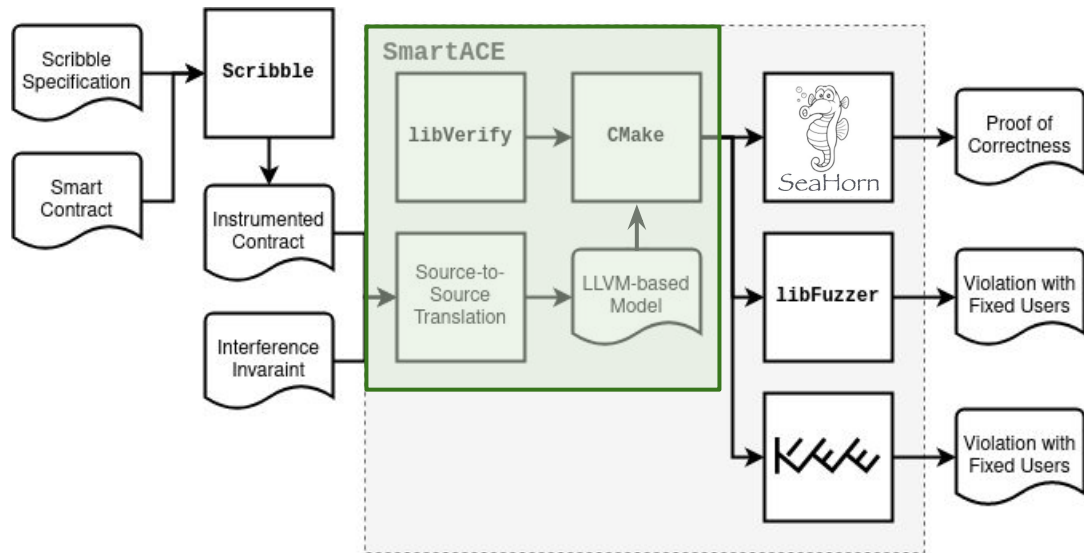


Overview: SmartACE Architecture

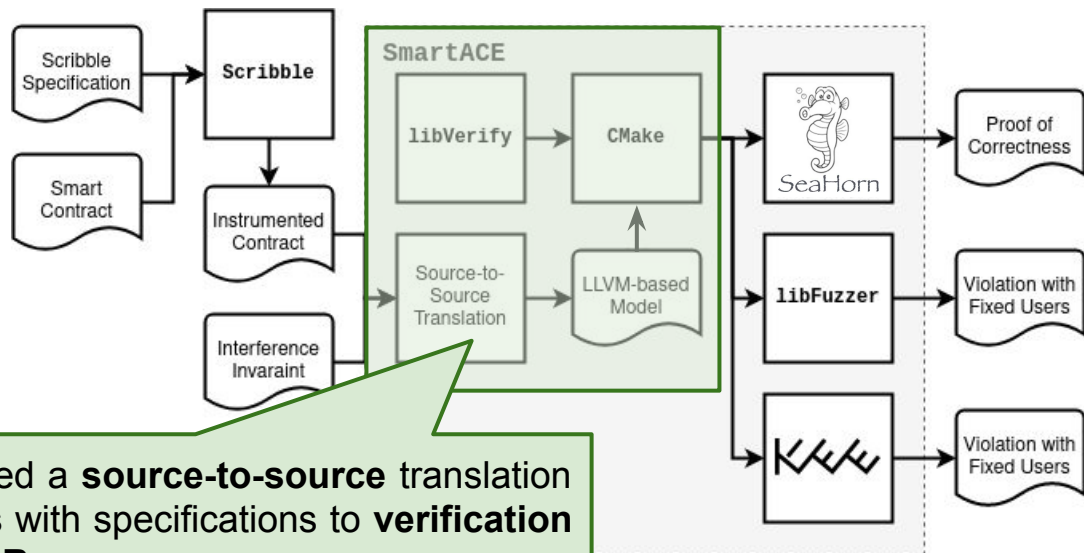


High-level specifications (e.g., *contract invariants*) are provided via the Scribble specification language. Scribble converts all specifications to assertions, so **SmartACE can also verify unannotated contracts** with one or more assertions.

Overview: SmartACE Architecture



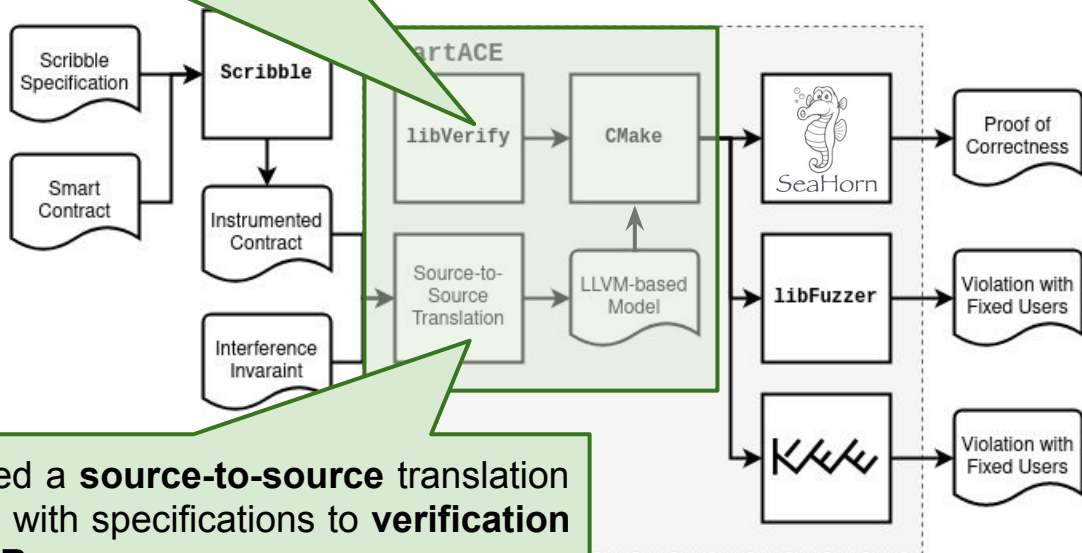
Overview: SmartACE Architecture



We have implemented a **source-to-source** translation from smart contracts with specifications to **verification problems in LLVM-IR**.

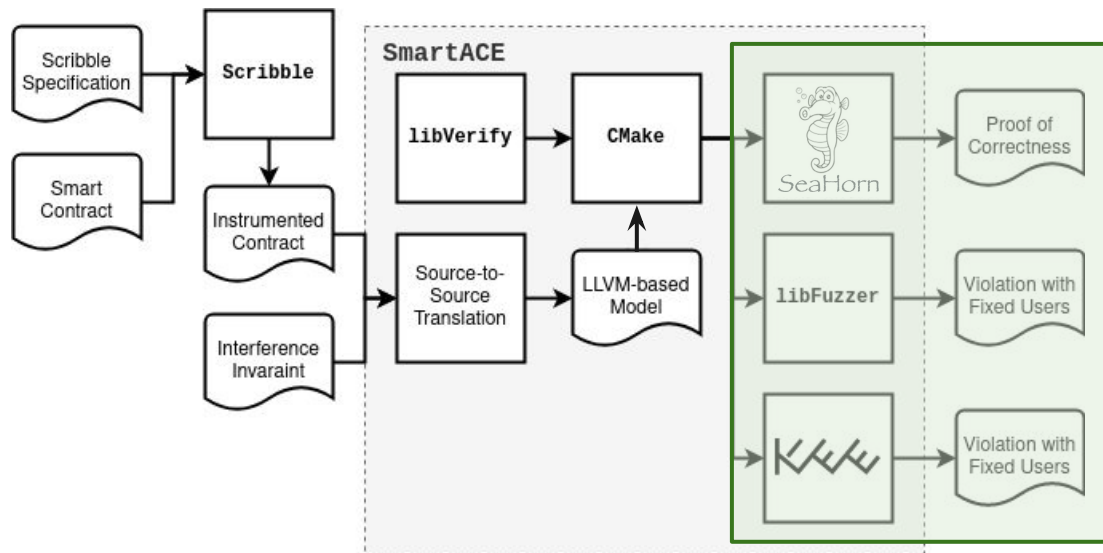
A **novel runtime** called `libVerify` decouples the source-to-source-translation from the integration details of each analyzer.

Architecture

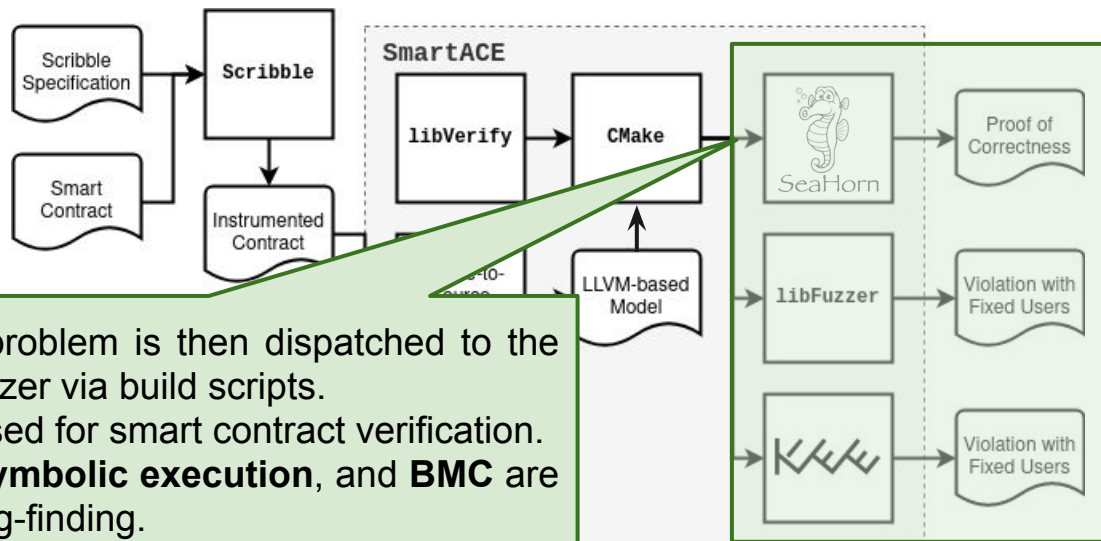


We have implemented a **source-to-source** translation from smart contracts with specifications to **verification problems in LLVM-IR**.

Overview: SmartACE Architecture



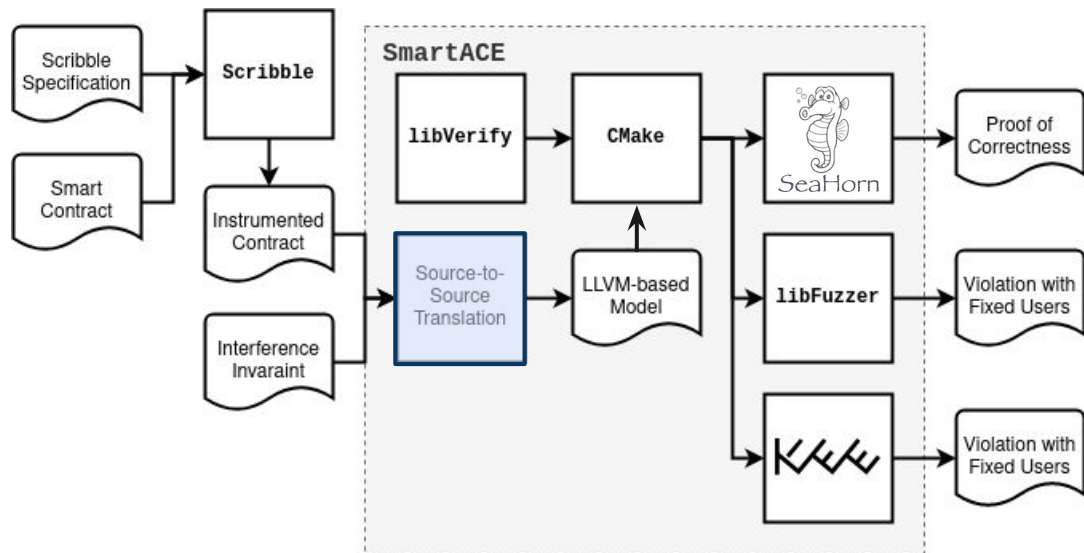
Overview: SmartACE Architecture



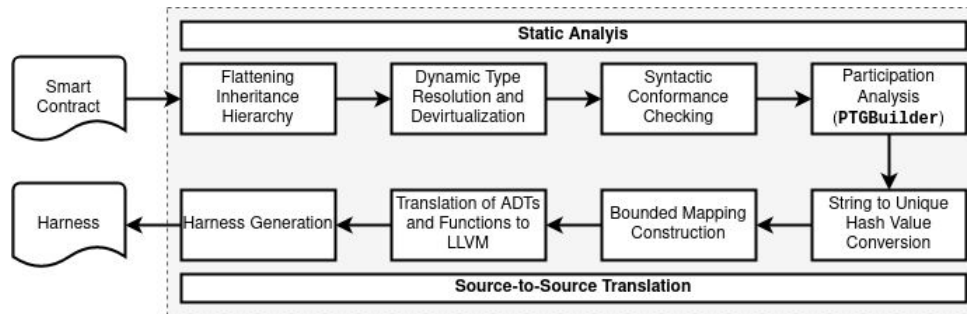
The verification problem is then dispatched to the appropriate analyzer via build scripts.

1. **PCMC** is used for smart contract verification.
2. **Fuzzing, symbolic execution, and BMC** are used for bug-finding.

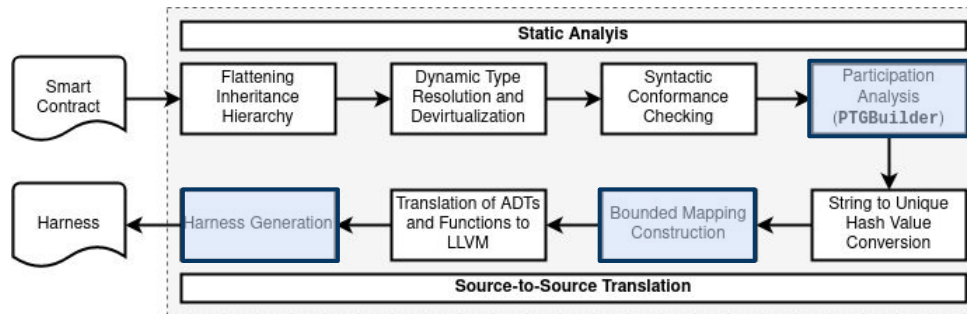
Overview: SmartACE Architecture



Conversion: Source-to-Source Translation Pipeline



Conversion: Source-to-Source Translation Pipeline

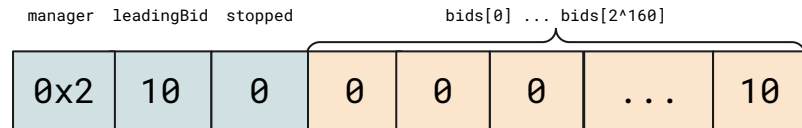


Conversion: Participation Analysis

```

01. contract Auction {
02.     mapping(address => uint) bids;
03.     address manager; uint leadingBid; bool stopped;
04.
05.
06.     function bid() public payable {
07.         // Checks permission to place bid.
08.         require(!stopped, "Auction must be ongoing.");
09.         require(msg.sender != manager, "Bidder must not be manager.");
10.         // Checks bid is valid.
11.         uint curBid = bids[msg.sender];
12.         require(curBid == 0 || curBid < leadingBid, "Leading bidder.");
13.         require(curBid + msg.value > leadingBid, "Bid too small.");
14.         // Records bid.
15.         bids[msg.sender] = curBid + msg.value;
16.         leadingBid = bids[msg.sender];
17.     }
18.
19.     constructor(address manager) public { /* ... */ }
20.     function withdraw() public { /* ... */ }
21.     function stop() public { /* ... */ }
22. }

```



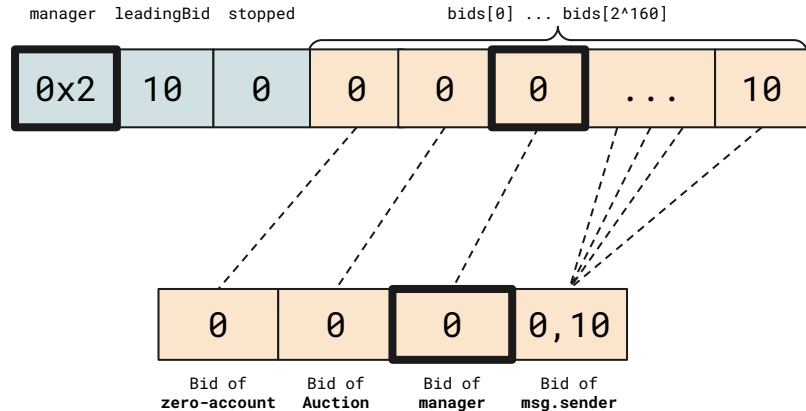
Participation analysis is used to determine a set of representative users that are sufficient for smart contract PCMC.

Conversion: Participation Analysis

```

01. contract Auction {
02.   mapping(address => uint) bids;
03.   address manager; uint leadingBid; bool stopped;
04.
05.
06.   function bid() public payable {
07.     // Checks permission to place bid.
08.     require(!stopped, "Auction must be ongoing.");
09.     require(msg.sender != manager, "Bidder must not be manager.");
10.     // Checks bid is valid.
11.     uint curBid = bids[msg.sender];
12.     require(curBid == 0 || curBid < leadingBid, "Leading bidder.");
13.     require(curBid + msg.value > leadingBid, "Bid too small.");
14.     // Records bid.
15.     bids[msg.sender] = curBid + msg.value;
16.     leadingBid = bids[msg.sender];
17.   }
18.
19.   constructor(address manager) public { /* ... */ }
20.   function withdraw() public { /* ... */ }
21.   function stop() public { /* ... */ }
22. }

```



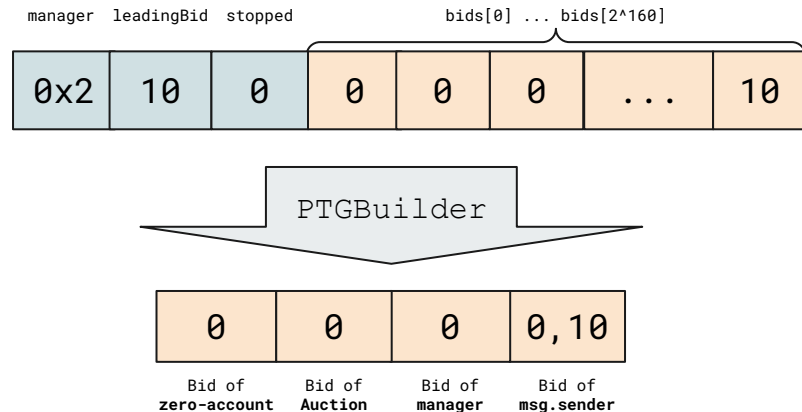
Participation is inferred via static analysis, and is used to group users based on the nature of their participation. PCMC enables verification using only a representative user from each group.

Conversion: Participation Analysis

```

01. contract Auction {
02.   mapping(address => uint) bids;
03.   address manager; uint leadingBid; bool stopped;
04.
05.
06.   function bid() public payable {
07.     // Checks permission to place bid.
08.     require(!stopped, "Auction must be ongoing.");
09.     require(msg.sender != manager, "Bidder must not be manager.");
10.     // Checks bid is valid.
11.     uint curBid = bids[msg.sender];
12.     require(curBid == 0 || curBid < leadingBid, "Leading bidder.");
13.     require(curBid + msg.value > leadingBid, "Bid too small.");
14.     // Records bid.
15.     bids[msg.sender] = curBid + msg.value;
16.     leadingBid = bids[msg.sender];
17.   }
18.
19.   constructor(address manager) public { /* ... */ }
20.   function withdraw() public { /* ... */ }
21.   function stop() public { /* ... */ }
22. }

```



Participation analysis is performed automatically using the PTGBuilder algorithm introduced in our **SAS 2021** paper.

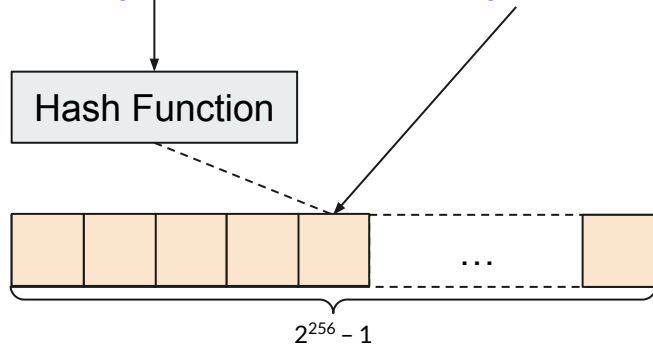


Conversion: Mapping Representation

Conversion: Mapping Representation

Mappings in Solidity

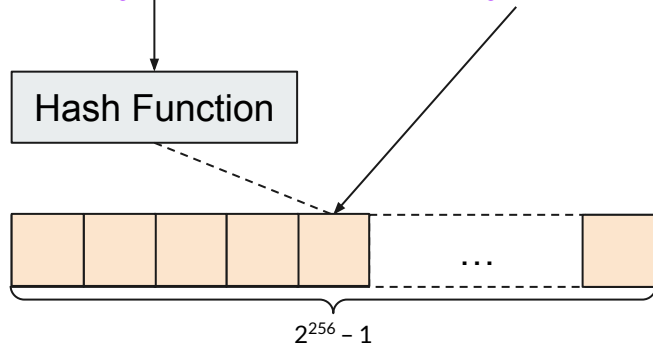
```
bids[msg.sender] = curBid + msg.value;
```



Conversion: Mapping Representation

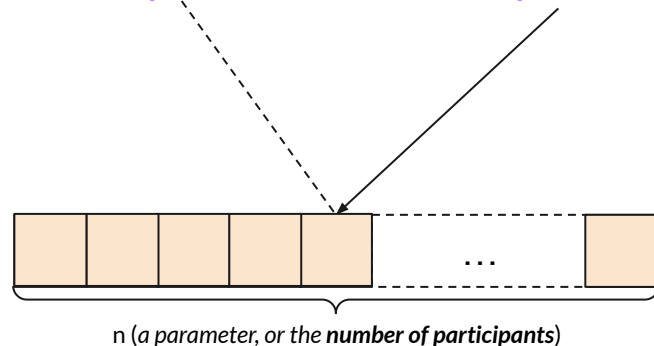
Mappings in Solidity

```
bids[msg.sender] = curBid + msg.value;
```

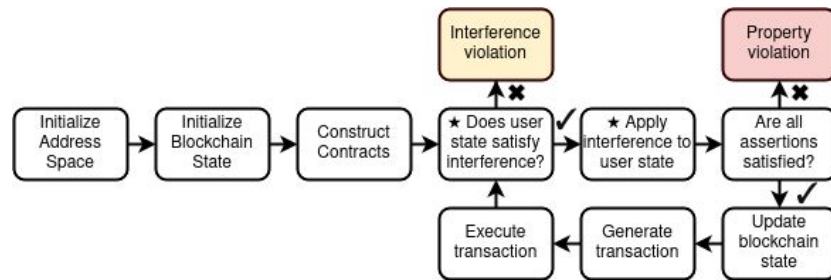


Mappings in SmartACE

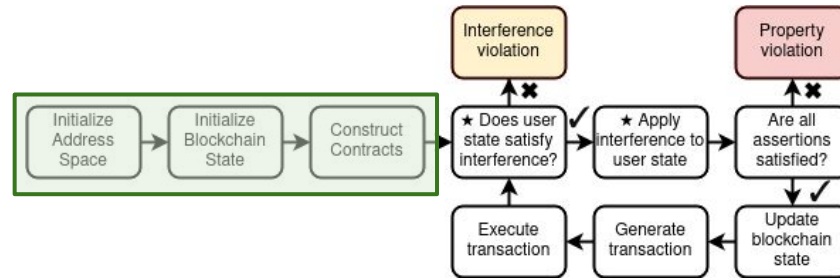
```
bids[msg.sender] = curBid + msg.value;
```



Conversion: Execution Model (*Harness Design*)



Conversion: Execution Model (*Harness Design*)



Contract construction is performed as outlined at the start of this presentation:

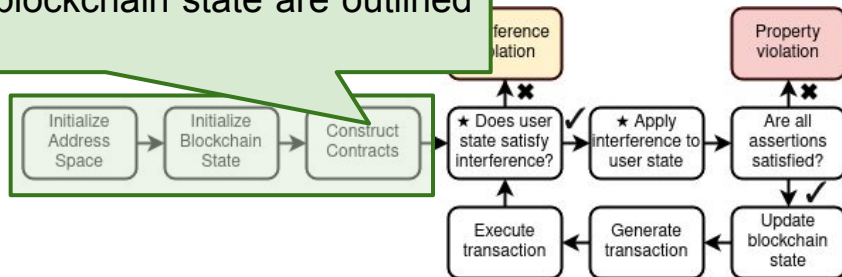
0x0	0	0	0	0	...	0
-----	---	---	---	---	-----	---

`constructor(0x5), msg.sender = 0x10, msg.value = 0`

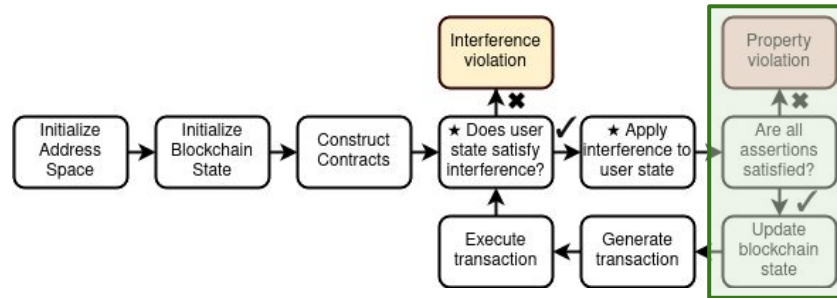
0x5	0	0	0	0	...	0
-----	---	---	---	---	-----	---

The address space and blockchain state are outlined in the full paper.

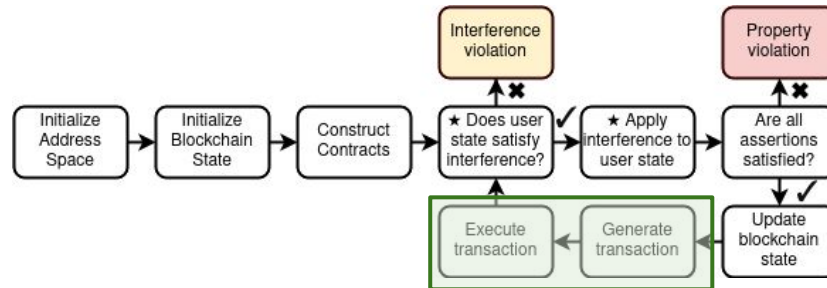
(Harness Design)



Conversion: Execution Model (*Harness Design*)



Conversion: Execution Model (*Harness Design*)



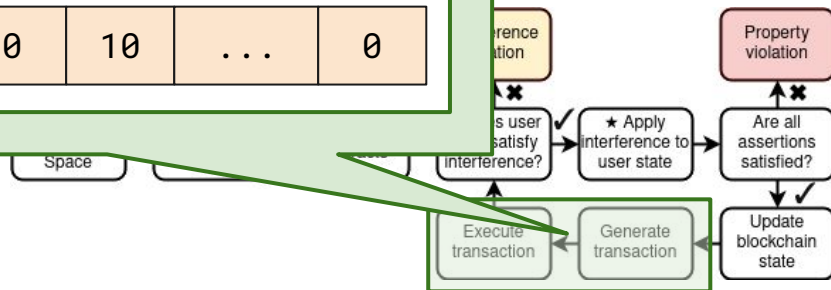
Transactions are generated and executed as outlined at the start of this presentation:

0x5	0	0	0	0	...	0
-----	---	---	---	---	-----	---

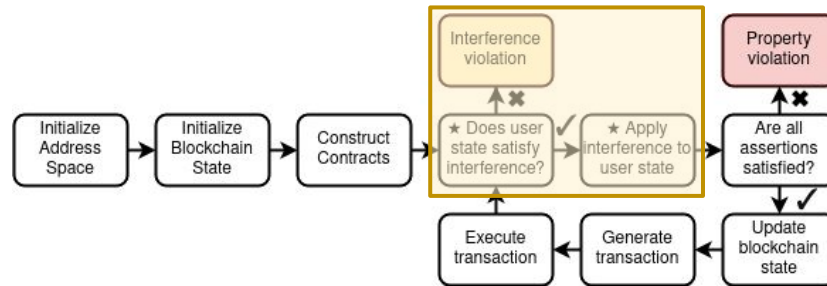
`bid(), msg.sender = 0x1, msg.value = 10`

0x5	10	0	0	10	...	0
-----	----	---	---	----	-----	---

(Harness Design)

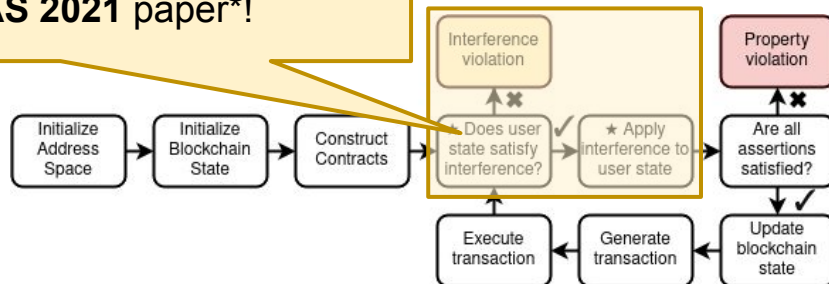


Conversion: Execution Model (*Harness Design*)



Conversion: Execution Model (*Harness Design*)

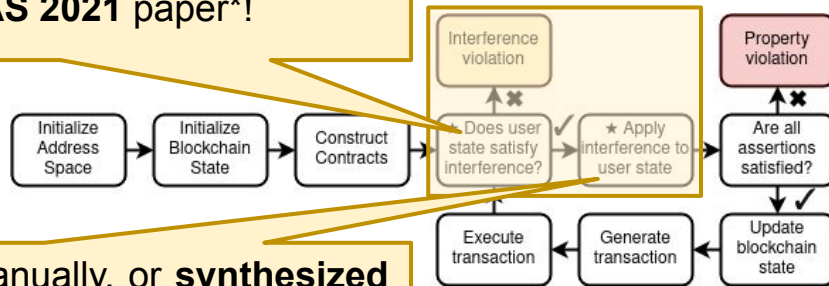
Interference enables verification using **only a few representative participants**. This technique was introduced in our **SAS 2021** paper*!



* Wesley et al., "Compositional verification of smart contracts through communication abstraction", 28th International Symposium in Static Analysis, 2021, pp. 429–452.

Conversion: Execution Model (*Harness Design*)

Interference enables verification using **only a few representative participants**. This technique was introduced in our **SAS 2021** paper*!



Interference is provided manually, or **synthesized automatically** by SeaHorn. Note that interference synthesis is **not reducible to sequential program verification** and required a novel solution.

* Wesley et al., "Compositional verification of smart contracts through communication abstraction", 28th International Symposium in Static Analysis, 2021, pp. 429–452.



Limitations: Unsupported Features and Future Work

Standard assumptions:

1. Inline assembly is not supported.
2. Reentrancy is not modeled (*best practices prevent reentrancy and can be checked syntactically*).
3. Each contract-typed variable corresponds to a single call to [new](#).

Current limitations of smart contract PCMC:

4. Mapping indices are addresses whereas mapping values are numeric (*nested mappings are fine*).
5. Addresses may only be compared for **(dis)equality**.

Unimplemented features:

6. Gas is not modeled and assumed to be unlimited.
7. Low-level features such as cryptography and block numbers are not supported.
8. Calls to contract interfaces with **unknown specifications** are not supported.
9. Time is monotonic, but **is not guaranteed to increase**.

Case Studies



Overview: Case Study Design

Goals of the case study:

1. Evaluate the effectiveness of SmartACE and Scribble to verify real-world smart contracts.
2. Identify challenges in applying SmartACE or Scribble to real-world smart contracts.
3. Study the impact of parameterization on the efficiency of SmartACE.



Overview: Case Study Design

Goals of the case study:

1. Evaluate the effectiveness of SmartACE and Scribble to verify real-world smart contracts.
2. Identify challenges in applying SmartACE or Scribble to real-world smart contracts.
3. Study the impact of parameterization on the efficiency of SmartACE.

Choice of contracts:

- An implementation of both Ownable and RefundEscrow from OpenZeppelin.
 - **Reason:** OpenZeppelin is a popular and widely used and open-source Solidity library.
- A simplified implementation of an open bid auction contract.
 - **Reason:** Open bid auctions lead to novel specifications regarding the largest bid placed so far.



Properties: Specifications in Scribble

Scribble supports **function annotations** (*post-conditions on a specific method*) and **contract invariants**.

However, Scribble does not support temporal properties of the form:

*“If **function** $x()$ has been called at least once, then [some property holds].”*

However, such properties can be instrumented using standard techniques, as outlined in our paper.



Properties: Specifications in Scribble

Scribble supports **function annotations** (*post-conditions on a specific method*) and **contract invariants**.

However, Scribble does not support temporal properties of the form:

“If function $x()$ has been called at least once, then [some property holds].”

However, such properties can be instrumented using standard techniques, as outlined in our paper.

Many of the properties verified **could be expressed** using the aforementioned features, e.g.:

`RefundEscrow`: *“If `close` has been called, then `enableRefunds` has not been called.”*

Several properties were novel in smart contract verification and required additional instrumentation.



Properties: Universal Map Quantifiers

The correctness of many smart contracts depends on specifications quantified over mappings, e.g.:

Auction: *“Any pair of distinct nonzero bids are unequal.”*



Properties: Universal Map Quantifiers

The correctness of many smart contracts depends on specifications quantified over mappings, e.g.:

Auction: *“Any pair of distinct nonzero bids are unequal.”*

Our insight to encode this problem.

Desired Property: *“For any k users, [some property holds].”*

Equivalent Property: *“**There does not exist** k users such that [some property **does not** hold].”*



Properties: Universal Map Quantifiers

The correctness of many smart contracts depends on specifications quantified over mappings, e.g.:

Auction: *“Any pair of distinct nonzero bids are unequal.”*

Our insight to encode this problem.

Desired Property: *“For any k users, [some property holds].”*

Equivalent Property: *“**There does not exist** k users such that [some property **does not** hold].”*

The property is instrumented by **selecting k arbitrary users** at contract construction, and then **searching for a counterexample using these k users**. This is easy to instrument at the level of Solidity source code.



Properties: Aggregate Properties

Aggregate properties are also common in smart contract verification, e.g.:

RefundEscrow: *“If close hasn’t been called, then the **sum of all deposits** is the escrow balance.”*



Properties: Aggregate Properties

Aggregate properties are also common in smart contract verification, e.g.:

RefundEscrow: *“If close hasn’t been called, then the **sum of all deposits** is the escrow balance.”*

Automated reasoning about aggregate properties requires **custom proof rules**. For example, a sum property can be verified using a **ghost variable that tracks the sum** of the mapping. This is common in prior work (e.g., VeriSmart, solc-verify, VerX, Solythesis, SolTypes).



Properties: Aggregate Properties

Aggregate properties are also common in smart contract verification, e.g.:

RefundEscrow: *“If close hasn’t been called, then the **sum of all deposits** is the escrow balance.”*

Automated reasoning about aggregate properties requires **custom proof rules**. For example, a sum property can be verified using a **ghost variable that tracks the sum** of the mapping. This is common in prior work (e.g., VeriSmart, solc-verify, VerX, Solythesis, SolTypes).

In our case studies, we show that **sum aggregates are compatible with local reasoning**. We also consider the **max aggregate properties not seen in prior work**.

Auction: *“The maximum bid equals the leading bid.”*



Properties: Aggregate Properties

Aggregate properties are also common in smart contract verification, e.g.:

RefundEscrow: *“If close hasn’t been called, then the **sum of all deposits** is the escrow balance.”*

Automated reasoning about aggregate properties requires **custom proof rules**. For example, a sum property can be verified using a **ghost variable that tracks the sum** of the mapping. This is common in prior work (e.g., VeriSmart, solc-verify, VerX, Solythesis, SolTypes).

In our case studies, we show that **sum aggregates are compatible with local reasoning**. We also consider the **max aggregate properties not seen in prior work**.

Auction: *“The maximum bid equals the leading bid.”*

Let’s take a look at how this proof rule works!



Properties: Max Aggregation

```
01. contract Auction {
02.   mapping(address => uint) bids;
03.   address manager; uint leadingBid; bool stopped;
04.   uint _max = 0; bool _monotonic = true;
05.
06.   function bid() public payable {
07.     uint _pre = bids[msg.sender];
08.     // Checks permission to place bid.
09.     require(!stopped, "Auction must be ongoing.");
10.     require(msg.sender != manager, "Bidder must not be manager.");
11.     // Checks bid is valid.
12.     uint curBid = bids[msg.sender];
13.     require(curBid == 0 || curBid < leadingBid, "Leading bidder.");
14.     require(curBid + msg.value > leadingBid, "Bid too small.");
15.     // Records bid.
16.     bids[msg.sender] = curBid + msg.value;
17.     leadingBid = bids[msg.sender];
18.     uint _post = bids[msg.sender];
19.     if (_max < _post) { _max = _post; }
20.     if (_post < _pre) { _monotonic = false; }
21.   }
22. }
```

Properties: Max Aggregation

```
01. contract Auction {
02.     mapping(address => uint) bids;
03.     address manager; uint leadingBid; bool stopped;
04.     uint _max = 0; bool _monotonic = true;
05.
06.     function bid() public payable {
07.         uint _pre = bids[msg.sender];
08.         // Checks permission to place bid.
09.         require(!stopped, "Auction must be ongoing.");
10.         require(msg.sender != manager, "Bidder must not be manager.");
11.         // Checks bid is valid.
12.         uint curBid = bids[msg.sender];
13.         require(curBid == 0 || curBid < leadingBid, "Leading bidder.");
14.         require(curBid + msg.value > leadingBid, "Bid too small.");
15.         // Records bid.
16.         bids[msg.sender] = curBid + msg.value;
17.         leadingBid = bids[msg.sender];
18.         uint _post = bids[msg.sender];
19.         if (_max < _post) { _max = _post; }
20.         if (_post < _pre) { _monotonic = false; }
21.     }
22. }
```

Variables are added to track the maximum bid recorded so far, along with whether the maximum bid is monotonically increasing.

Properties: Max Aggregation

```

01. contract Auction {
02.   mapping(address => uint) bids;
03.   address manager; uint leadingBid; bool stopped;
04.   uint _max = 0; bool _monotonic = true;
05.
06.   function bid() public payable {
07.     uint _pre = bids[msg.sender];
08.     // Checks permission to place bid.
09.     require(!stopped, "Auction must be ongoing.");
10.     require(msg.sender != manager, "Bidder must not be manager.");
11.     // Checks bid is valid.
12.     uint curBid = bids[msg.sender];
13.     require(curBid == 0 || curBid < leadingBid, "Leading bidder.");
14.     require(curBid + msg.value > leadingBid, "Bid too small.");
15.     // Records bid.
16.     bids[msg.sender] = curBid + msg.value;
17.     leadingBid = bids[msg.sender];
18.     uint _post = bids[msg.sender];
19.     if (_max < _post) { _max = _post; }
20.     if (_post < _pre) { _monotonic = false; }
21.   }
22. }

```

Variables are added to track the maximum bid recorded so far, along with whether the maximum bid is monotonically increasing.

After each bid, the maximum bid is updated. If the new bid is smaller than the previous bid, then the monotonic flag is set to false.

Properties: Max Aggregation

```

01. contract Auction {
02.     mapping(address => uint) bids;
03.     address manager; uint leadingBid; bool stopped;
04.     uint _max = 0; bool _monotonic = true;
05.
06.     function bid() public payable {
07.         uint _pre = bids[msg.sender];
08.         // Checks permission to place bid.
09.         require(!stopped, "Auction must be ongoing.");
10.         require(msg.sender != manager, "Bidder must not be manager.");
11.         // Checks bid is valid.
12.         uint curBid = bids[msg.sender];
13.         require(curBid == 0 || curBid < leadingBid, "Leading bidder.");
14.         require(curBid + msg.value > leadingBid, "Bid too small.");
15.         // Records bid.
16.         bids[msg.sender] = curBid + msg.value;
17.         leadingBid = bids[msg.sender];
18.         uint _post = bids[msg.sender];
19.         if (_max < _post) { _max = _post; }
20.         if (_post < _pre) { _monotonic = false; }
21.     }
22. }

```

Variables are added to track the maximum bid recorded so far, along with whether the maximum bid is monotonically increasing.

After each bid, the maximum bid is updated. If the new bid is smaller than the previous bid, then the monotonic flag is set to false.

Then `_max == max(bids)`, so long as `_monotonic == true`.



Results: Performance

Benchmark		Verification		Bug Finding ($n = 5$)			Bug Finding ($n = 500$)		
Contract	Prop.	Manual (s)	Auto. (s)	BMC (s)	FUZ (s)	SYM (s)	BMC (s)	FUZ (s)	SYM (s)
Ownable	O1	1	1	1	1	90	1	1	85
	O2	1	1	1	1	25	1	1	27
	O3	1	1	1	1	25	1	1	27
RefundEscrow	R1	2	2	2	1	454	140	22	—
	R2	2	3	2	2	5	277	32	3124
	R3	2	7	5	26	5	1800	74	—
	R4	12	17	3	6	90	1724	296	—
	R5	3	4	3	2	6	2010 ^(†)	33	—
Auction	A1	9	59	2	4	39	564	21	123
	A2	69	246	4	3	533	4392	397	—

Results: Performance

Benchmark		Verification		Bug Finding ($n = 5$)			Bug Finding ($n = 500$)		
Contract	Prop.	Manual (s)	Auto. (s)	BMC (s)	FUZ (s)	SYM (s)	BMC (s)	FUZ (s)	SYM (s)
Ownable	O1	1	1	1	1	90	1	1	85
	O2	1	1	1	1	25	1	1	27
	O3	1	1	1	1	25	1	1	27
RefundEscrow	R1	2	2	2	1	454	140	22	—
	R2	2	3	2	2	5	277	32	3124
	R3	2	7	5	26	5	1800	74	—
	R4	12	17	3	6	90	1724	296	—
	R5	3	4	3	2	6	2010 ^(†)	33	—
Auction	A1	9	59	2	4	39	564	21	123
	A2	69	246	4	3	533	4392	397	—

All properties were instrumented and verified successfully using PCMC.

Results: Performance

Benchmark		Verification		Bug Finding ($n = 5$)			Bug Finding ($n = 500$)		
Contract	Prop.	Manual (s)	Auto. (s)	BMC (s)	FUZ (s)	SYM (s)	BMC (s)	FUZ (s)	SYM (s)
Ownable	O1	1	1	1	1	90	1	1	85
	O2	1	1	1	1	25	1	1	27
	O3	1	1	1	1	25	1	1	27
RefundEscrow	R1	2	2	2	1	454	140	22	—
	R2	2	3	2	2	5	277	32	3124
	R3	2	7	5	26	5	1800	74	—
	R4	12	17	3	6	90	1724	296	—
	R5	3	4	3	2	6	2010 ^(†)	33	—
Auction	A1	9	59	2	4	39	564	21	123
	A2	69	246	4	3	533	4392	397	—

Fault-injection demonstrated that SmartACE is also able to detect real bugs.

Results: Performance

Benchmark		Verification		Bug Finding ($n = 5$)			Bug Finding ($n = 500$)		
Contract	Prop.	Manual (s)	Auto. (s)	BMC (s)	FUZ (s)	SYM (s)	BMC (s)	FUZ (s)	SYM (s)
Ownable	O1	1	1	1	1	90	1	1	85
	O2	1	1	1	1	25	1	1	27
	O3	1	1	1	1	25	1	1	27
RefundEscrow	R1	2	2	2	1	454	140	22	—
	R2	2	3	2	2	5	277	32	3124
	R3	2	7	5	26	5	1800	74	—
	R4	12	17	3	6	90	1724	296	—
	R5	3	4	3	2	6	2010 ^(†)	33	—
Auction	A1	9	59	2	4	39	564	21	123
	A2	69	246	4	3	533	4392	397	—

Bug-finding via BMC and SYM were made feasible through parameterization.

Results: Performance

Benchmark		Verification		Bug Finding ($n = 5$)			Bug Finding ($n = 500$)		
Contract	Prop.	Manual (s)	Auto. (s)	BMC (s)	FUZ (s)	SYM (s)	BMC (s)	FUZ (s)	SYM (s)
Ownable	O1	1	1	1	1	90	1	1	85
	O2	1	1	1	1	25	1	1	27
	O3	1	1	1	1	25	1	1	27
RefundEscrow	R1	2	2	2	1	454	140	22	—
	R2	2	3	2	2	5	277	32	3124
	R3	2	7	5	26	5	1800	74	—
	R4	12	17	3	6	90	1724	296	—
	R5	3	4	3	2	6	2010 ^(†)	33	—
Auction	A1	9	59	2	4	39	564	21	123
	A2	69	246	4	3	533	4392	397	—

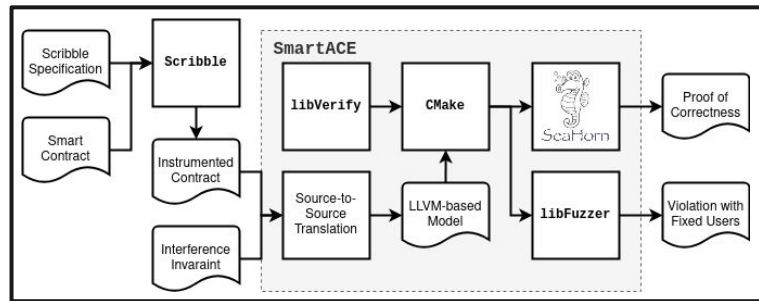
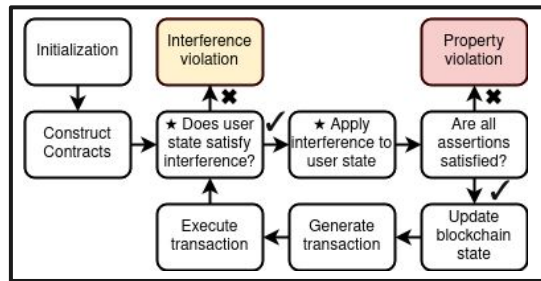
All but one property could be verified using “default” interference.

Results: Performance

Benchmark		Verification		Bug Finding ($n = 5$)			Bug Finding ($n = 500$)		
Contract	Prop.	Manual (s)	Auto. (s)	BMC (s)	FUZ (s)	SYM (s)	BMC (s)	FUZ (s)	SYM (s)
Ownable	O1	1	1	1	1	90	1	1	85
	O2	1	1	1	1	25	1	1	27
	O3	1	1	1	1	25	1	1	27
RefundEscrow	R1	2	2	2	1	454	140	22	—
	R2	2	3	2	2	5	277	32	3124
	R3	2	7	5	26	5	1800	74	—
	R4	12	17	3	6	90	1724	296	—
	R5	3	4	3	2	6	2010 ^(†)	33	—
Auction	A1	9	59	2	4	39	564	21	123
	A2	69	246	4	3	533	4392	397	—

In this case, interference synthesis was able to find adequate interference.

Conclusion



SmartACE is a **smart contract analysis framework** with a focus on **parameterized verification**.

SmartACE supports **model checking** (SeaHorn), **fuzzing** (libFuzzer), and **symbolic execution** (Klee), with the **potential to extend** to other C program analyzers.

SmartACE uses participation analysis and bounded mappings to facilitate parameterized analysis.

We have illustrated the effectiveness of SmartACE by **verifying contracts from the OpenZeppelin** library. Additional case studies can be found in our **SAS2021** paper.

SmartACE and Experiments are available on GitHub: <https://github.com/contract-ace>